

Artificial Intelligence - Report Lab 8 & 9

Anh Khoi Pham - ITCSIU23018

May 29, 2026

1 Perceptron

The Perceptron is a simple supervised learning model used for binary classification. It receives numerical input features, computes a weighted sum with a bias term, and then applies an activation function to produce the predicted class label.

```
1  class Perceptron:
2      def __init__(self, learning_rate=0.01,
3                    weights=None, n_iters=100):
4          self.learning_rate = learning_rate
5          self.n_iters = n_iters
6          self.activation_func = self._unit_step_function
7          self.weights = weights
8
9      def _unit_step_function(self, x):
10         return np.where(x > 0, 1, 0)
11
12
13     def fit(self, X, y):
14         x_samples, n_features = X.shape
15
16         X = np.c_[np.ones(x_samples), X]
17
18         if self.weights is None:
19             self.weights = np.zeros(n_features +
20                                     1)
21
22         for _ in range(self.n_iters):
23             for idx, x_i in enumerate(X):
24                 linear_output = np.dot(x_i, self.weights)
25                 y_predicted = self.activation_func(linear_output)
26
27                 update = self.learning_rate * (y[idx] - y_predicted)
28                 self.weights += update * x_i
29
30
```

```
31     def predict(self, X):
32         X = np.c_[np.ones(X.shape[0]), X]
33         linear_output = np.dot(X, self.weights)
34         y_predicted = self.activation_func(
35             linear_output)
36         return y_predicted
```

This Perceptron model contains three main methods for predicting the label:

Fit

The `fit()` method trains the model using the input data `X` and labels `y`. It first adds a bias column to the input, initializes the weights if needed, and then updates the weights repeatedly based on the prediction error.

Predict

The `predict()` method uses the learned weights to classify new input samples. It adds the bias term, calculates the linear output using the dot product, and returns the final class labels after applying the activation function.

Activation Function

The `_unit_step_function()` converts the linear output into a binary prediction. If the output is greater than 0, it returns 1; otherwise, it returns 0.

2 Train and Inference

We don't want our model to be overfitted, so we need to split the data into 2 parts: train and test

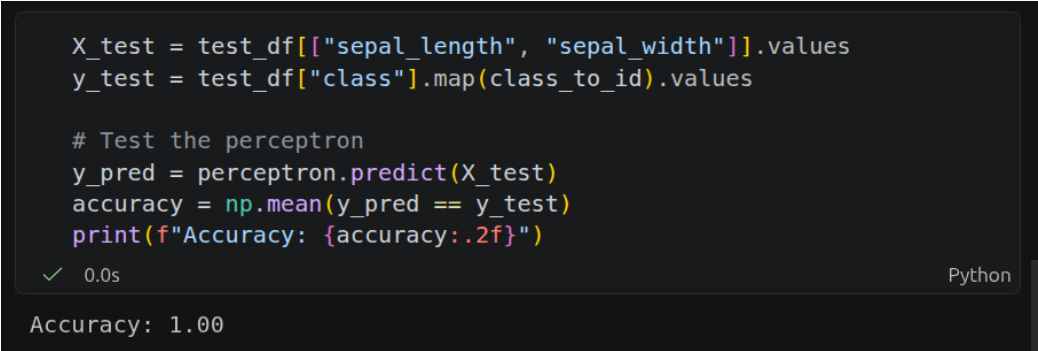
```
1  # Split the dataset into training and testing
   sets
2  train_df = df2.sample(frac=0.8, random_state=42)
3  test_df = df2.drop(train_df.index)
```

Then prepare the train dataset, and call the `fit()` method from the model to update the weights.

```
1 X_train = train_df[["sepal_length", "sepal_width"]].values
2 y_train = train_df["class"].map(class_to_id).values
3
4 # Train the perceptron
5 perceptron = Perceptron(learning_rate=0.01)
6 perceptron.fit(X_train, y_train)
```

Finally, the `predict()` method will use the fitted weights to classify the input from the test dataset.

```
1 X_test = test_df[["sepal_length", "sepal_width"]].values
2 y_test = test_df["class"].map(class_to_id).values
3
4 # Test the perceptron
5 y_pred = perceptron.predict(X_test)
6 accuracy = np.mean(y_pred == y_test)
7 print(f"Accuracy: {accuracy:.2f}")
```



```
X_test = test_df[["sepal_length", "sepal_width"]].values
y_test = test_df["class"].map(class_to_id).values

# Test the perceptron
y_pred = perceptron.predict(X_test)
accuracy = np.mean(y_pred == y_test)
print(f"Accuracy: {accuracy:.2f}")
```

✓ 0.0s

Python

Accuracy: 1.00

3 Multiclass Perceptron

Implement a Perceptron algorithm to classify multiple classes.

```
1 class MultiClassPerceptron:
2     def __init__(self, learning_rate=0.01,
3                   n_iters=1000):
4         self.learning_rate = learning_rate
```

```
4         self.n_iters = n_iters
5         self.classifiers = {}
6
7
8     def fit(self, X, y):
9         self.classes = np.unique(y)
10
11         for cls in self.classes:
12             binary_y = np.where(y == cls, 1, 0)
13             clf = Perceptron(learning_rate=self.
14                             learning_rate, n_iters=self.
15                             n_iters)
16             clf.fit(X, binary_y)
17             self.classifiers[cls] = clf
18
19     def predict(self, X):
20         class_scores = np.zeros((X.shape[0], len(
21             self.classes)))
22
23         for idx, cls in enumerate(self.classes):
24             clf = self.classifiers[cls]
25             class_scores[:, idx] = np.dot(np.c_[
26                 np.ones(X.shape[0]), X], clf.
27                 weights)
28
29         predicted_classes = np.argmax(
30             class_scores, axis=1)
31         return self.classes[predicted_classes]
```

This implementation follows the One-versus-All strategy for multiclass classification. For each class, the algorithm creates a binary classification problem where samples from that class are labeled as 1 and all other samples are labeled as 0. A separate `Perceptron` classifier is then trained for each class and stored in `self.classifiers`.

During prediction, each trained classifier computes a score for the input sample. The model compares all class scores and chooses the class with the highest score using `argmax`. Therefore, the final multiclass prediction is the class whose binary classifier is most confident for the given input.